

Programación reactiva con Spring WebFlux (Mono y Flux)

Qué es y qué problema resuelve

WebFlux resuelve el problema de atender muchas peticiones a la vez con muy pocos hilos. En vez de manejar solo 1 hilo, el modelo reactivo devuelve de inmediato una receta de cómo obtener el dato y esa receta solo se ejecuta cuando alguien se suscribe. Mientras un dato tarda en llegar, el hilo del event loop queda libre para atender otras peticiones.

El paquete trae dos ejercicios separados:

- 01-webflux-mono — un solo valor por respuesta: qué es un Mono, la diferencia entre "vacío" y "no encontrado", y por qué bloquear dentro de un controlador reactivo hunde toda la aplicación, no solo esa petición.
- 02-webflux-flux — varios valores en el tiempo: un sensor que emite una lectura por segundo con Flux, y cómo la misma fuente de datos se puede servir como JSON completo (espera a que termine) o como stream de eventos (llega dato por dato).

Dónde se ve en el código

- **01-webflux-mono/.../repo/EmployeeRepository.java** — la versión reactiva devuelve Mono<Employee> con delayElement(5s) para simular latencia de red sin depender de una base de datos real; también trae findByIdBloqueante(...), una versión que sí bloquea, a propósito.
- **01-webflux-mono/.../rest/EmployeeRestController.java** — los endpoints "correctos": manejo de vacío con switchIfEmpty(), y errores con onErrorResume().
- **01-webflux-mono/.../rest/BloqueanteRestController.java** — literalmente "lo que no hay que hacer": el mismo controlador de un proyecto MVC anterior, copiado tal cual dentro de una app reactiva, para medir el daño con scripts/comparar.sh.
- **01-webflux-mono/.../rest/HiloRestController.java** — expone qué hilo atendió la petición; llamado varias veces seguidas, muestra que siempre son los mismos 4-8 nombres (el *event loop*), no un hilo nuevo por petición.
- **02-webflux-flux/.../service/SensorService.java** — Flux.interval(1s).map(...): un flujo infinito, algo que un List<T> no puede representar.
- **02-webflux-flux/.../rest/LecturaRestController.java** — el mismo Flux servido de dos formas (APPLICATION_JSON vs TEXT_EVENT_STREAM), más operadores sobre un flujo vivo: filter(), takeUntil(), collectList().

Qué pasa si no lo usas

- Bloquear dentro de un endpoint reactivo (como hace `BloqueanteRestController`, con `Thread.sleep()` sobre el *event loop*) no solo hace lenta esa petición: duerme uno de los pocos hilos disponibles (4-8), `scripts/comparar.sh` mide justo esta diferencia lanzando N peticiones simultáneas contra la versión reactiva y la bloqueante.
- Confundir "vacío" con "no encontrado": un Mono vacío responde 200 sin cuerpo, no 404. Sin `switchIfEmpty()` explícito, un cliente que busca un empleado que no existe recibe una respuesta "exitosa" vacía en vez de un error claro.
- Servir un Flux infinito como JSON normal sin `take(n)`: Spring intenta juntar todos los elementos antes de responder
- Usar `Map.of(...)` para el cuerpo de una respuesta con más de un campo: el orden de iteración de `Map.of()` se aleatoriza en cada arranque de la JVM; el JSON de salida sale con los campos en distinto orden cada vez que se reinicia la app, rompiendo cualquier cosa que dependa de ese orden (como un cut en un script). Por eso los controladores usan `record` en vez de `Map`.

Cómo correrlo

```
cd 01-webflux-mono && ./mvnw spring-boot:run
./scripts/probar.sh      # recorre los endpoints (otra terminal)
./scripts/comparar.sh 50 # compara reactivo vs bloqueante con 50 peticiones simultáneas
```

Para el proyecto de Flux (usa `curl -N` para no perder el streaming):

```
cd 02-webflux-flux && ./mvnw spring-boot:run
curl -N http://localhost:8075/api/lecturas/stream
```